

CRYPTOGRAPHY

& INFORMATION SYSTEM SECURITY



Modul praktikum: Security System

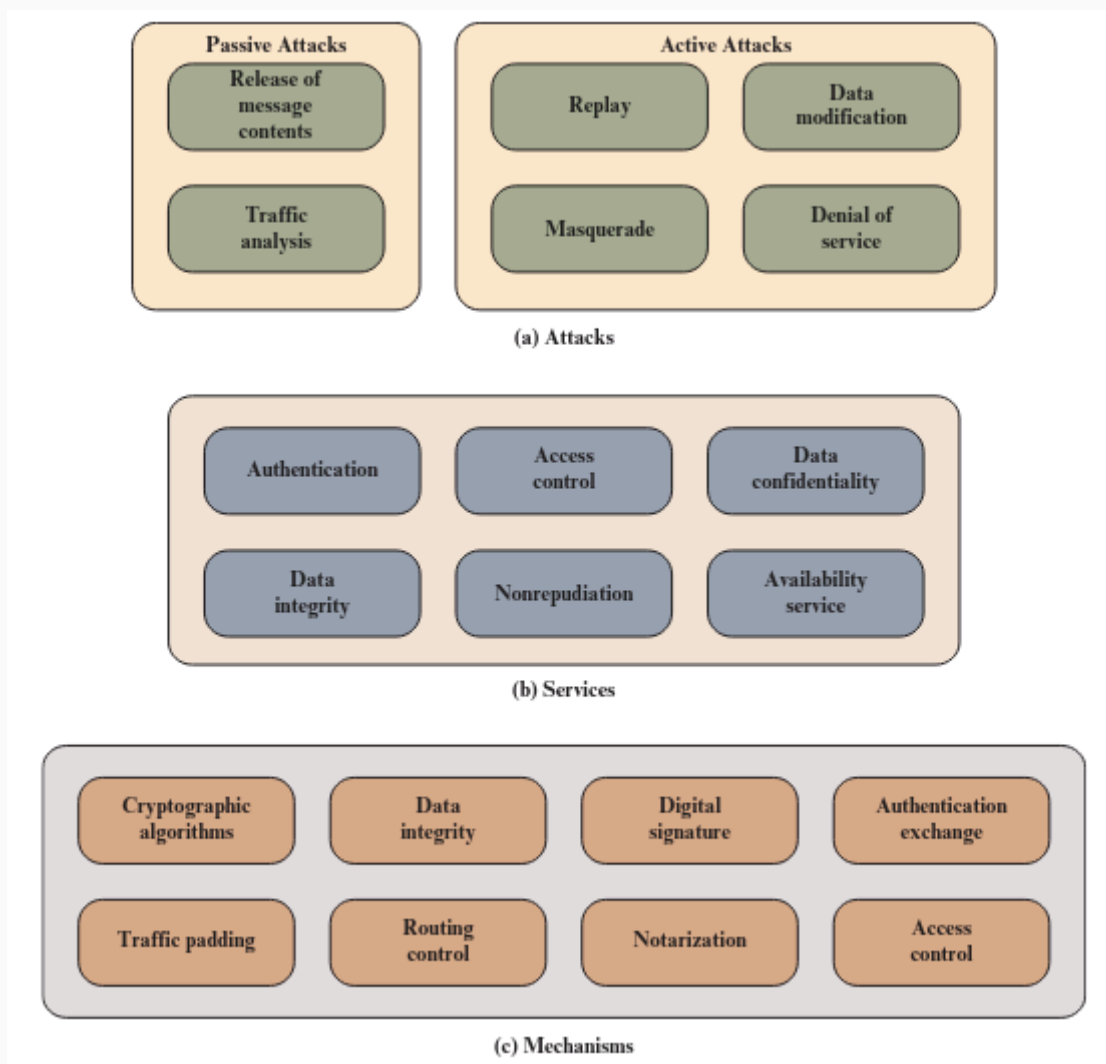
Daftar Isi

Daftar Isi.....	2
Overview Security System.....	3
Web Pembayaran Sederhana.....	3
Overview flow.....	3
Analisis Kebutuhan.....	4
Konsep Implementasi.....	4
User Authentication.....	4
Access Control List.....	6
Communication Security.....	7
Hubungan Mekanisme dan Security Service.....	7
Perkenalan OWASP.....	7
Penetration Testing.....	7
SQL Injection.....	7
Intro to JWT.....	7
Challenge.....	8
Challenge 1 (Exposed HASH).....	8
Challenge 2 (SQL Injection).....	8
Challenge 3 (IDOR).....	9
Challenge 4 (JWT).....	9
Challenge 5 (Brute Force).....	10
Tugas.....	11
Daftar Pustaka.....	12
Tim Asisten Praktikum.....	12

Overview Security System

Menurut (Stallings, 2023), **Security Attack** adalah segala tindakan yang membahayakan keamanan informasi yang dimiliki organisasi. **Security mechanism** adalah proses yang didesain untuk mendeteksi, mencegah, dan me-recover dari Security Attack. **Security Service** adalah proses atau layanan komunikasi yang meningkatkan keamanan sistem pemrosesan data dan transfer informasi suatu organisasi. Layanan dimaksudkan untuk menangkal security attack dan menggunakan satu atau lebih security mechanism untuk mendukung layanan. Konsep tersebut digambarkan pada Gambar 1.

Gambar 1. Key Concepts in Security
Sumber: Stallings, 2023



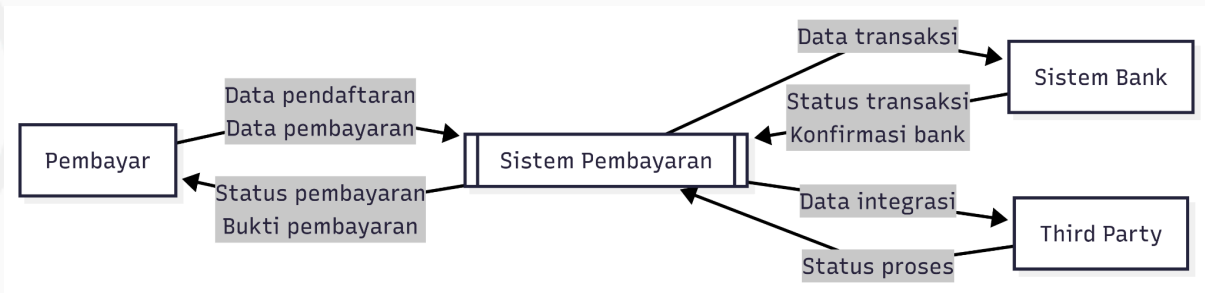
Web Pembayaran Sederhana

Overview flow

Sama seperti sebelumnya, Tim Anda sedang berburu dengan waktu sehingga tidak berikan dokumen ERD, tetapi hanya dokumen [DFD](#). Sistem ini adalah sistem pembayaran yang digunakan sebagai API untuk bertransaksi dari satu sistem ke opsi-opsi bank yang disediakan. Berdasarkan Gambar 2, Sistem pembayaran melibatkan pembayar, sistem bank, dan third party. Pembayar dapat melakukan transaksi dengan bank melalui sistem pembayaran. Data yang dibutuhkan terbatas

pada data registrasi, pembyaran, dan transaksi. Detail sistem dapat dilihat pada DFD level 1 dan 2 pada drive.

Gambar 2. DFD Level 0 Sistem Pembayaran



Analisis Kebutuhan

Berdasarkan DFD yang telah dibuat sebelumnya dan pengetahuan pada modul-modul sebelumnya, dibutuhkan beberapa Security Mechanism untuk memberikan Security Service yang lebih baik. Beberapa mekanisme tersebut adalah penerapan enkripsi, hashing, digital signature, dan HTTPS.

Tabel 1 menunjukkan daftar kebutuhan yang ada pada sistem dan keperluan mekanisme keamanannya. Tabel 1 hanya berisi contoh saja untuk memetakan pikiran. Untuk draftingnya, bisa masuk ke dalam ranah specification requirement, policy, dan sebagainya sesuai dengan relevansinya.

Tabel 1. Contoh daftar kebutuhan mekanisme keamanan

Kebutuhan	Mekanisme Keamanan	Alasan
User	Hashing: Bcrypt, User Authentication: ?	Perlu autentikasi agar tidak sembarang orang dapat memodifikasi data personal masing-masing.
Komunikasi HTTP	PKI: TLS	Dalam tahap deploy, banyak data yang tidak boleh dibaca sembarang orang dalam jaringan.
Pembayaran	User Authentication, Access Control: ?	Selain perlu autentikasi, tidak setiap orang dapat mengakses seluruh data. User diberikan akses terbatas terhadap data tertentu.

Konsep Implementasi

User Authentication

Kebutuhan autentikasi berbeda-beda pada setiap data/sistemnya. Misal untuk masuk ke aplikasi, kredensial autentikasinya berbasis Knowledge dengan data berupa email dan password. Implementasi sederhananya ditunjukkan oleh algoritma 1.

Algoritma 1. Contoh implementasi email dan password

```
import bcrypt
import sqlite3

def init_db():
    conn = sqlite3.connect("users.db")
    conn.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY,
```

```

        email TEXT UNIQUE,
        password_hash TEXT
    )
    """
    conn.commit()
    return conn

def register_user(conn, email, password):
    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt())
    try:
        conn.execute(
            "INSERT INTO users (email, password_hash) VALUES (?, ?)",
            (email, hashed)
        )
        conn.commit()
        print("Registrasi berhasil")
    except sqlite3.IntegrityError:
        print("Email sudah terdaftar")

def login_user(conn, email, password):
    row = conn.execute(
        "SELECT password_hash FROM users WHERE email = ?",
        (email,)
    ).fetchone()
    if not row:
        print("Login gagal")
        return
    if bcrypt.checkpw(password.encode(), row[0]):
        print("Login berhasil")
    else:
        print("Login gagal")

if __name__ == "__main__":
    conn = init_db()
    register_user(conn, "user@example.com", "password123")
    login_user(conn, "user@example.com", "password123")
    login_user(conn, "user@example.com", "wrongpassword")

```

Note : Kebutuhan autentikasi berbeda-beda pada setiap data dan sistemnya. Untuk masuk ke aplikasi misalnya, kredensial autentikasinya berbasis knowledge dengan data berupa email dan password. Algoritma 1 menunjukkan contoh implementasinya menggunakan Python dengan bcrypt sebagai fungsi hash password.

Implementasi tersebut merupakan implementasi dasar yang rentan untuk ditembus. Tabel 2 menunjukkan contoh daftar kebutuhan untuk autentikasi ini. Implementasinya perlu diuji apakah sudah mengatasi kerentanan yang mungkin terjadi.

Tabel 2. Contoh daftar kebutuhan autentikasi akses aplikasi

Kebutuhan	Kriteria
Data Kredensial	Unique email dan password. Database hanya menyimpan hasil bcrypt password, tidak secara raw.
Rate Limit	5 kali, interval akhir kelipatan 30 menit
Verifikasi email dan mekanisme pemulihan	Untuk pemulihan dibatasi hanya bisa untuk reset password.
Session lifetime	30 hari semenjak akses terakhir
Authentication Flow	1. Registrasi jika belum pernah. 2. Verifikasi email jika belum pernah. 3. Verifikasi password.. 4. Simpan session.

Notes:

- ISO 29115 sudah memberikan framework untuk Entity Authentication Assurance..
- Autentikasi digunakan dalam Access Control, tidak satu konsep, tetapi bisa satu modul.

Berdasarkan analisis sebelumnya, autentikasi tidak hanya diperlukan untuk akses aplikasi. Misal untuk setiap pembayaran, perlu autentikasi lain. Contoh implementasinya kredensial berbasis knowledge berupa PIN. Apabila diimplementasikan untuk di-consume di aplikasi mobile, dapat diberikan alternatif berbasis Inherence berupa sidik jari. Implementasi lebih rigid tidak ditunjukkan disini.

Untuk autentikasi ke sistem bank, standar harus ditetapkan. Standar dapat ditetapkan oleh pihak bank itu sendiri, pihak sistem pembayaran, ataupun kesepakatan bersama. Implementasi lebih rigid tidak ditunjukkan disini.

Access Control List

Salah satu contoh access control adalah Direct Access Control atau DAC (Maymí & Harris, 2022). Dalam konteks sistem ini, DAC tidak begitu tepat untuk digunakan karena harus per pengguna, sedangkan data penggunanya banyak. Maka dari itu, alternatifnya adalah mengelompokkannya berdasarkan role atau RBAC. Access tidak dibatasi berdasarkan data, tetapi modul, sehingga contoh kriterianya ditunjukkan pada Tabel 3. Untuk implementasinya, dapat dijadikan modul access control middleware bagi tiap modul lainnya.

Tabel 3. Contoh daftar hak akses

Role	Hak akses
Pembayar	Sistem Transaksi
Bank	Sistem Monitoring: Transaksi
Third Party	Sistem Monitoring: Transaksi
Admin	Sistem Monitoring: Pendaftaran Bank
Unauthenticated	Sistem Pendaftaran

Penerapan hak akses tidak dibatasi hanya per modul. Dua role bisa saja memiliki akses yang berbeda terhadap modul yang sama. Contohnya bank dan third party hanya diberi akses lihat terhadap dua data yang berbeda. Misal, third-party seperti e-commerce ternyata memiliki hak akses transaksi dan monitoring. Namun berbeda dengan pembayar, third-party hanya dapat memberikan akses bagi pembayar untuk melakukan transaksi. Di sisi lain, sistem monitoring bagi third-party adalah melihat data transaksi non-pengguna, bank yang digunakan, serta pemasukan yang didapat dari transaksi, sedangkan bank hanya dapat akses melihat data transaksi non-pengguna dan third-party yang terhubung.

Communication Security

Contoh penerapan keamanan ini adalah TLS dan sudah pernah diterapkan pada pertemuan sebelumnya.

Hubungan Mekanisme dan Security Service

Ambil contoh mekanisme RBAC. RBAC merupakan mechanism yang menyediakan service berupa access control, authentication (prasyarat akibat dibutuhkannya identity), juga data confidentiality. Jika dibutuhkan, RBAC dapat diintegrasikan dengan mechanism routing control. Contohnya role tertentu tidak dapat mengakses routes tertentu.

Perkenalan OWASP

OWASP atau Open Worldwide Application Security Project adalah foundation yang bersifat nonprofit yang bekerja untuk mengembangkan keamanan perangkat lunak (OWASP, t.t.-a). Di situsnya, OWASP memberikan informasi statistik berupa [10 besar Risiko Paling Kritis Keamanan Web Application](#) (OWASP, t.t.-b). Informasi ini memudahkan security untuk memberikan fokus awal dalam hal security. OWASP juga memberikan [framework sendiri untuk memudahkan testing](#).

Penetration Testing

Setelah security mechanism dirancang dan diimplementasikan, perlu ada tahap verifikasi bahwa implementasinya benar-benar berfungsi sesuai rencana. Penetration testing adalah pendekatan sistematis untuk melakukan verifikasi tersebut, bukan sebagai tujuan utama, melainkan sebagai alat konfirmasi bahwa sistem yang sudah dibangun tahan terhadap eksploitasi yang umum terjadi. Bedakan penetration testing dengan hacking, pentesting itu merupakan subset dari hacking sendiri karena hacking itu cakupannya lebih luas

SQL Injection

SQL injection terjadi ketika input user disisipkan langsung ke dalam query SQL tanpa validasi atau escaping. Perhatikan query berikut.

```
SELECT * FROM users WHERE email = '{email}' AND password = '{password}'
```

Jika field email diisi dengan nilai ' OR '1'='1' --, logika WHERE menjadi selalu bernilai true karena kondisi '1'='1' tidak pernah salah dan sisa query setelahnya dikomentari. Server akan mengembalikan baris pertama di tabel tanpa memverifikasi password sama sekali.

```
SELECT * FROM users WHERE email = '' OR '1'='1' --'
```

Untuk List-list SQLi yang lengkap bisa dicek disini :

<https://github.com/payload-box/sql-injection-payload-list>

Atau cari aja di google "sql injection payloads cheatsheet"

Intro to JWT

JSON Web Token atau JWT adalah format token yang membawa klaim antar dua pihak dalam satu string. Strukturnya terdiri dari tiga bagian yang dipisahkan titik: header, payload, dan signature.

Header menyimpan metadata termasuk algoritma yang digunakan untuk membuat signature. Payload menyimpan data seperti user ID dan role. Signature memverifikasi bahwa token tidak dimodifikasi oleh pihak lain.

Contoh token: eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoidXNlciJ9.xxxxx

Ketiganya bisa di-decode di jwt.io hanya dengan paste token di kolom Encoded. Perlu dicatat bahwa decode bukan decrypt, siapapun bisa membaca isi payload tanpa kunci apapun, sehingga payload tidak boleh berisi data sensitif yang belum terenkripsi.

Challenge

Pada praktikum ini disediakan beberapa challenge berikut untuk mensimulasikan celah yang umum ditemukan pada web pembayaran, mengacu pada OWASP Top 10. Tools yang akan digunakan disini adalah :

1. browser DevTools
2. Curl (buat yang linux, atau kalau mau pake windows curl.exe tapi syntax nya beda dikit)
3. jwt.io
4. crackstation.net.
5. python

Challenge 1 (Exposed HASH)

URL : <http://20.212.111.212:5000/challenge/1>

Persiapan: login ke aplikasi, lalu buka Developer Tools di browser (F12) dan buka tab Network.

Langkah-langkah:

1. Lakukan request ke endpoint profil user. Bisa lewat browser setelah login, atau lewat curl:

```
curl http://20.212.111.212:5000/api/users/alice
```

2. Perhatikan JSON response yang dikembalikan server di panel Response atau di output curl.
3. Cari field seperti `password_hash` atau `hashed_password` di response body.
4. Salin nilai hash tersebut, formatnya itu md5 (keliatan bentuknya kayak gitu kalau sering liat).
5. Buka crackstation.net, paste hash, dan submit.

Output yang diharapkan: crackstation mengembalikan plaintext password apabila password yang digunakan termasuk dalam wordlist umum.

Celah yang terjadi bukan pada algoritma hash-nya. md5 memang tahan brute force secara komputasi, tetapi hash yang seharusnya tidak pernah meninggalkan server justru dikirim ke client di setiap API response. Ini terjadi karena API mengembalikan raw object dari database tanpa filtering field.

Rekomendasi perbaikannya adalah mendefinisikan schema response secara eksplisit, misalnya dengan serializer atau DTO, sehingga field seperti `password_hash` tidak pernah muncul di response manapun.

Challenge 2 (SQL Injection)

URL: <http://20.212.111.212:5000/challenge/2>

Persiapan: buka halaman challenge, siapkan curl atau gunakan form login di browser. Sebelum mencoba, baca pengantar SQL Injection di atas.

Langkah-langkah:

1. Coba login normal dulu dengan username "guest" dan password "guest123" untuk memastikan form berjalan.
2. Coba login dengan username berikut dan password sembarang:

admin' --

3. Atau via curl:

```
curl -X POST http://20.212.111.212:5000/api/login -H "Content-Type: application/json" -d '{"username": "admin'\''--", "password": "apapun"}'
```

4. Perhatikan response yang dikembalikan.

Output yang diharapkan: server mengembalikan response login sukses sebagai admin tanpa password yang benar.

Query yang dikonstruksi dengan string concatenation memungkinkan karakter `--` mengomentari sisa query, sehingga pengecekan password dilewati sepenuhnya.

Rekomendasi perbaikannya adalah menggunakan parameterized query sehingga input user tidak pernah diinterpretasikan sebagai SQL.

Challenge 3 (IDOR)

URL: <http://20.212.111.212:5000/challenge/3>

Persiapan: buka halaman challenge. Halaman sudah menampilkan transaksi milik alice (user_id = 1) dengan ID 1, 2, dan 3.

Langkah-langkah:

1. Perhatikan transaction ID yang tampil di panel "My Transactions".
2. Gunakan input fetch di halaman untuk mencoba ID lain, atau via curl:

```
curl http://20.212.111.212:5000/api/transactions/11
```

3. Amati apakah server mengembalikan data yang bukan milik alice.

Output yang diharapkan: server mengembalikan detail transaksi milik user lain.

IDOR atau Insecure Direct Object Reference terjadi ketika server hanya memverifikasi bahwa user sudah login, tetapi tidak memverifikasi apakah resource yang diminta milik user tersebut. Ini masuk kategori A01:2025 Broken Access Control pada OWASP Top 10.

Rekomendasi perbaikannya adalah menambahkan filter user_id pada setiap query, di mana user_id diambil dari token yang sudah diverifikasi server, bukan dari input user.

Challenge 4 (JWT)

URL: <http://20.212.111.212:5000/challenge/4>

Persiapan: buka halaman challenge, login dengan username "user" dan password "user123". Sebelum mencoba, baca pengantar JWT di atas.

Langkah-langkah:

1. Login lewat form di halaman. Token JWT akan tampil di halaman setelah login berhasil.
2. Buka jwt.io, paste token ke kolom Encoded.
3. Di bagian header decoded, ubah nilai "alg" dari "HS256" menjadi "none".
4. Di bagian payload, ubah nilai "role" menjadi "admin".
5. Salin token dari kolom Encoded, lalu hapus bagian setelah titik kedua sehingga formatnya menjadi `header.payload`. dengan titik di akhir tanpa signature.
6. Gunakan token yang sudah dimodifikasi untuk memanggil endpoint admin lewat panel di halaman, atau via curl:

```
curl http://20.212.111.212:5000/api/admin -H "Authorization: Bearer [MODIFIED_TOKEN]"
```

Output yang diharapkan: server menerima token tanpa signature dan mengembalikan data admin.

Celah alg:none terjadi karena server membaca nilai algoritma dari header token itu sendiri, bukan dari konfigurasi server.

Rekomendasi perbaikannya adalah hardcode algoritma yang diterima di sisi server sebagai allowlist, dan menggunakan library JWT yang menolak alg:none secara default.

Challenge 5 (Brute Force)

URL: <http://20.212.111.212:5000/challenge/5>

Persiapan: Python sudah terinstal.

Langkah-langkah:

1. Klik tombol Login di halaman. Nilai header X-Session-Data akan tampil di halaman.
2. Salin hex string tersebut, lalu jalankan script berikut:

```
ciphertext_hex = "PASTE_HEX_DISINI"
ciphertext = bytes.fromhex(ciphertext_hex)

for key in range(256):
    decrypted = bytes([b ^ key for b in ciphertext])
    try:
        result = decrypted.decode("utf-8")
        if result.isprintable():
            print(f"Key {key:3d} (0x{key:02x}): {result}")
    except Exception:
        pass
```

3. Cari output yang menghasilkan teks bermakna.

Output yang diharapkan: satu key menghasilkan JSON yang terbaca.

XOR dengan single-byte key hanya memiliki 256 kemungkinan sehingga brute force selesai dalam milidetik.

Rekomendasi perbaikannya adalah tidak mengirim state apapun yang sensitif ke client dalam bentuk terenkripsi, cukup simpan di server dan kirim session ID yang opaque.

Tugas

URL : <http://20.212.111.212>

Mari kita cobain CTF gaes, anak ilkom wajib mencoba berburu bendera di mata kuliah cyber. Ok ini ketentuannya yah bagi yang belum tau CTF :

1. buat akun di platform. Buka ke platform CTF nya: <http://20.212.111.212>, klik Register, isi username, email (email palsu juga gapapa gaada verif), dan password, lalu login. Tidak ada verifikasi email, akun langsung aktif setelah register.
2. Setelah login, daftar challenge tersedia di menu Challenges. Setiap challenge punya kolom submit flag. Flag berformat UPI{...} dan ditemukan dengan mengeksploitasi aplikasi target di <http://20.212.111.212:5001>. Kerjakan challenge dalam urutan apapun, tidak ada ketergantungan antar challenge.
3. Untuk writeup, tidak ada format khusus. Yang wajib dicantumkan hanya **nama lengkap, nama account di platform CTF nya dan NIM** di bagian atas dokumen. Isi minimal per challenge yang berhasil diselesaikan adalah celah apa yang ditemukan, **bagaimana cara menemukannya, langkah eksploitasi** yang dilakukan, **dan rekomendasi perbaikan**. Tulis juga **"Flag : <flagnyah> "** Di bagian bawah sebelum challenge selanjutnya. Dikumpulkan dalam bentuk PDF dengan format **NIM_writeup.pdf**
4. Good luck have fun

Deadline:

C1: 11/05/2026 23:59 WIB

C2: 12/05/2026 23:59 WIB

[Link Pengumpulan](#)

Daftar Pustaka

Maymí, F., & Harris, S. (2022). *CISSP all-in-one exam guide* (Ninth edition). McGraw Hill.

OWASP. (t.t.-a). About the OWASP Foundation. *OWASP Foundation*. Diambil 3 Maret 2026, dari <https://owasp.org/about/>

OWASP. (t.t.-b). The Ten Most Critical Web Application Security Risks. *OWASP Foundation*. Diambil 3 Maret 2026, dari https://owasp.org/Top10/2025/0x00_2025-Introduction/

Stallings, W. (2023). *Cryptography and network security: Principles and practice* (Eighth edition, global edition). Pearson.

Tim Asisten Praktikum

1. Franklin
2. **Lyan (PIC Praktikum)**
3. Ichsan
4. Alvin
5. Radhi